

capacity and throughput. Similarly, containers can be removed when the demand falls. Using orchestration frameworks such as Kubernetes and Apache Mesos further simplifies elastic scaling.

- **Performance:** As compared to virtual machines, containers are very lightweight and have minimal impact on performance.
- **Hundred per cent availability:** With the deployment of multiple containers, redundancy can be built into the application. So, if any container fails, the other peers will take over to provide the service.

Container orchestration

Container orchestration is defined as the process of automated arrangement, coordination and the management of computer systems, middleware and services.

Container orchestration is linked with service-oriented architecture, virtualisation, provisioning, converged infrastructure and dynamic data centres. It is regarded as the process of aligning the business requests with the applications, data and infrastructure. It defines the policies and service levels via automated workflows, provisioning and change management. It creates a powerful application-aligned infrastructure that can be scaled up or down depending on application needs. Orchestration reduces the time and effort for deploying multiple instances of a single application.

Container orchestration tools offer powerful and effective solutions for creating, managing and updating multiple containers in coordination across multiple hosts. Orchestration allows the users to share data between services and process tasks in an asynchronous fashion.

Orchestration offers the following unique features:

- Provisioning hosts
- Instantiating a set of containers
- Rescheduling failed containers
- Linking containers together via simple administration-oriented GUI interfaces
- Exposing services to machines outside of the cluster
- Scaling in, out and down the cluster by adding or removing containers at any point of time

Consider an example in which IT software infrastructure comprises Apache, PHP, Python, Ruby, Node.js and React apps running on a few containers which are connected to a replicated DB. In such a case, there is no requirement for container orchestration. But when the application keeps on growing rapidly with time, it requires massive resources in terms of CPU, RAM, etc. That's when it needs to be split into smaller chunks, where every chunk is responsible for an independent task, maintained by respective teams like microservices. So, now there is a requirement for a caching layer, as well as a queuing system to increase performance, effectively synchronise the tasks and share data successfully among the services.

Various challenges come into this operational scenario, like service discovery, load balancing, configuration management, storage management, regular system health checks, auto-configuration of nodes and zero-downtime deployments.

Container orchestration provides effective and powerful solutions to meet various demands. There are many players in the market like Kubernetes, AWS ECS and Docker Swarm. So, it is important to compare the performance of Kubernetes against that of AWS ECS to assist IT sysadmins in selecting the better of the two.

Kubernetes

Kubernetes is an open source platform designed by Google and now maintained by the Cloud Native Computing Foundation. It is regarded as the most feature-rich and widely used orchestration framework for automation, scaling and operating application containers across clusters of hosts, and it provides container-centric infrastructure. It works with a range of container tools including Docker.

Kubernetes is designed to work in multiple environments including bare metal, on-premise VMs and public clouds.

With Kubernetes, one can perform the following operations:

- Orchestrate containers across multiple hosts.
- Make effective use of hardware to maximise resources to run all sorts of enterprise apps.
- Control and automate application deployments and updates.
- Mount and add storage to stateful apps.
- Scale containerised applications and all resources, on-the-fly.
- Manage all sorts of services, which guarantee that all deployed apps are running in the same way as they are deployed.

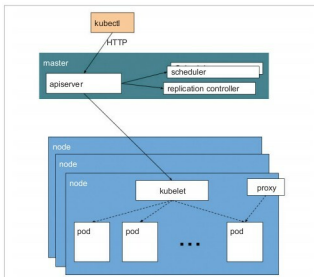


Figure 1: Kubernetes architecture and components